

Atividade Prática

Projeto CinelA (Recomendador de Filmes)

Objetivo

Criar uma interface simples que recomenda filmes com base nas preferências do usuário e em seu “estado de espírito”.

Estrutura da Interface (Fluxo de IHC)

Entrada: Usuário seleciona gêneros, tempo máximo e descreve o “mood”.

Processamento: O sistema monta um prompt estruturado e envia ao Gemini.

Feedback: Mostrar spinner (“Analisando catálogo...”) para indicar que está rodando.

Saída: Exibir recomendações com título, ano e justificativa.

Preparação do Ambiente de Desenvolvimento

1. Instalar o Python

Acesse python.org/downloads e baixe a versão **Python 3.10 ou superior**.

Durante a instalação, marque a opção **“Add Python to PATH”**. Isso facilitará rodar comandos no terminal.

2. Instalar o VS Code (Editor de Código)

Baixe em code.visualstudio.com.

O VS Code será usado para editar o código.

3. Criar uma Pasta específica para o Projeto

Crie uma pasta chamada cineia.

Dentro dela salvaremos mais adiante o arquivo do código em Python (ex: app.py).

4. Abrir o Terminal

No Windows: pressione Win + R, digite cmd e aperte Enter.

5. Instalar o Gerenciador de pacotes e as Bibliotecas Necessárias

Instale o pip (gerenciador oficial de pacotes do python)

Antes teste se já não está instalado

```
pip --version
```

Se aparecer algo como pip 24.0 from ... significa que já está instalado.

Caso não esteja instalado execute a linha

```
python -m ensurepip --upgrade
```

Instale as Bibliotecas por meio do PIP

No terminal, um comando por vez, digite e pressione a tecla “Enter”:

```
pip install streamlit
```

```
pip install google-generativeai
```

Estas linhas de comando instalarão as bibliotecas:

Streamlit

Utilizaremos para facilitar a implementação da interface web.

Google-generativeai

Utilizaremos para conectar nossa interface ao Gemini.

6. Criar o Arquivo do Projeto na linguagem Python

No VS Code, crie um arquivo chamado app.py e insira o código python (nosso CinelA) a seguir:

```
import streamlit as st
import google.generativeai as genai

# Configuração da chave (peça para os alunos gerarem no Google AI Studio)
genai.configure(api_key="SUA_CHAVE_AQUI")
model = genai.GenerativeModel('gemini-pro')

# Configuração da página
st.set_page_config(page_title="CinelA", page_icon="🎬")
st.title("🎬 CinelA: Seu Próximo Filme")
st.markdown("Descubra filmes baseados no seu estado de espírito atual.")

# --- INTERFACE ---
with st.sidebar:
    st.header("Preferências")
    genero = st.multiselect("Gêneros favoritos:",
                           ["Ação", "Drama", "Sci-Fi", "Comédia", "Terror", "Documentário"])
    tempo = st.slider("Duração máxima (minutos):", 60, 240, 120)

mood = st.text_area("Descreva como você está se sentindo ou o que busca no filme:",
```

```
placeholder="Ex: Quero um filme de ficção científica com reviravoltas na história.")
```

```
botao_recomendar = st.button("Buscar Recomendações")
```

```
# --- LÓGICA DE PROCESSAMENTO ---
```

```
if botao_recomendar:
```

```
    if not mood:
```

```
        st.warning("Por favor, descreva o que você deseja assistir!")
```

```
    else:
```

```
        with st.spinner("Analisando catálogo cinematográfico..."):
```

```
            prompt = f"""
```

```
            Você é um especialista em cinema.
```

```
            Recomende 3 filmes para alguém que gosta de {' '.join(genero)} e que durem até {tempo} minutos.
```

```
            O usuário descreveu o clima do filme como: '{mood}'.
```

```
            Para cada filme, forneça: Título, Ano e uma frase curta do porquê combina com o pedido.
            """
```

```
        try:
```

```
            response = model.generate_content(prompt)
```

```
            st.success("Aqui estão minhas sugestões:")
```

```
            st.markdown("---")
```

```
            st.write(response.text)
```

```
        except Exception as e:
```

```
            st.error(f"Erro ao conectar com a IA: {e}")
```

```
st.caption("Desenvolvido na disciplina Desenvolvimento Full Stack – Sistemas de Informação -  
Universidade Franciscana (UFN)")
```

7. Obter a Chave da API do Gemini

1. Acesse [Google AI Studio](#).
2. Faça login com sua conta Google.
3. Clique em **"Get API Key"** e copie a chave.
4. No código Python que implementamos (arquivo app.py), substitua "SUA_CHAVE_AQUI" pela chave gerada.

(Pense na chave como uma **senha** que permite ao seu programa conversar com o Gemini.)

8. Rodar o Projeto

No terminal, dentro da pasta cineia, execute a linha:

```
streamlit run app.py
```

Esta linha de comando abrirá automaticamente o navegador com a nossa interface do CinelA.

Caso isso não ocorra, após a execução da linha de comando, basta abrir <http://localhost:8501> no navegador.

Erro comum: Se aparecer “ModuleNotFoundError”, significa que as bibliotecas não foram instaladas corretamente.

9. Como Testar

1. Escolha alguns gêneros no menu lateral. (ex.: Drama, Comédia.)
2. Ajuste a duração máxima (ex.: 120 minutos).
3. Digite como você está se sentindo (ex: “quero uma comédia leve para relaxar”).
4. Clique em **Buscar Recomendações**.
5. O Gemini processará os dados e sugerirá filmes com título, ano e justificativa, com base nas informações preenchidas.

Erros Comuns

ModuleNotFoundError: Bibliotecas não instaladas corretamente.

Erro de chave: Verifique se substituiu "SUA_CHAVE_AQUI" pela chave correta.

Nada aparece: Confira se está rodando dentro da pasta cineia.

Algumas reflexões relacionadas com IHC

Prevenção de Erros: O que acontece se nenhum gênero for selecionado?

Carga Cognitiva: Slider e multiselect são melhores que texto livre?

Feedback: O spinner ajuda a reduzir ansiedade do usuário?

Vamos Expandir o projeto

Iremos implementar um mecanismo de **RLHF** (Reinforcement Learning from Human Feedback).

O que é RLHF?

É uma técnica usada para treinar modelos de IA com base em avaliações humanas. Em vez de apenas aprender com dados prontos, o modelo recebe feedback direto das pessoas sobre suas respostas, esse feedback é usado para ajustar o comportamento da IA.

Etapas de funcionamento:

1. **Geração de respostas:** o modelo produz várias opções de saída para uma mesma entrada.
2. **Avaliação humana:** pessoas classificam ou escolhem quais respostas são melhores.
3. **Treinamento com reforço:** o modelo é ajustado para aumentar a probabilidade de gerar respostas semelhantes às preferidas pelos humanos.
4. **Iteração contínua:** quanto mais feedback, mais o modelo se aproxima do que os usuários consideram útil, claro e adequado.

10. Aplicando RLHF ao CinelA

Imagine que o CinelA recomenda 3 filmes.

O usuário pode dar um **feedback simples**: “Gostei” ou “Não gostei”.

Esse feedback pode ser armazenado e usado para **ajustar o sistema**.

Com o tempo, o CinelA aprenderia quais tipos de recomendações agradam mais os usuários e passaria a sugerir filmes mais alinhados às preferências reais.

Como iremos incluir RLHF no projeto CinelA

Adicionar um campo de feedback no Streamlit.

Registrar esse feedback em um arquivo .csv.

Código python para coletar feedback:

```
col1, col2 = st.columns(2)
```

```
with col1:
```

```
    if st.button("👍 Gostei"):
```

```
        with open("feedback.csv", "a") as f:
```

```
            f.write(f"{mood},{genero},{tempo},Gostei\n")
```

```
        st.success("Obrigado pelo seu feedback positivo!")
```

```
with col2:
```

```
    if st.button("👎 Não gostei"):
```

```
        with open("feedback.csv", "a") as f:
```

```
            f.write(f"{mood},{genero},{tempo},Não gostei\n")
```

st.info("Feedback registrado. Vamos melhorar!")

O trecho de código para coletar feedback (código logo a cima) precisa ser inserido dentro do mesmo arquivo app.py, mas em uma posição estratégica para que o usuário só veja essa opção depois que as recomendações forem exibidas.

Onde colocar o código de feedback

No seu app.py, o ponto ideal é logo após o bloco que mostra as sugestões da IA (st.success(...), st.write(response.text)), dentro do try:.

Assim, o fluxo ficará assim:

1. O usuário pede recomendações.
2. O sistema mostra os filmes sugeridos.
3. Em seguida, aparece a opção de dar feedback.

Algumas Reflexões sobre RLHF

IHC: como o feedback melhora a experiência do usuário?

- **Feedback imediato:** o clique já registra a resposta.
- **Ícones universais:**  e  são compreendidos sem precisar ler muito.
- **Menor esforço cognitivo:** não há necessidade de abrir menu ou confirmar seleção.
- **Estética mais amigável:** aproxima a interface de padrões comuns em apps e redes sociais.

IA: como o feedback humano pode tornar o modelo mais confiável?

Ciência de Dados: como armazenar e analisar esses dados para melhorar o sistema?

Código completo do CineIA (Python)

```
import streamlit as st
```

```
import google.generativeai as genai
```

```
# Configuração da chave (os alunos devem gerar no Google AI Studio)
```

```
genai.configure(api_key="SUA_CHAVE_AQUI")
```

```
model = genai.GenerativeModel('gemini-pro')
```

```

# Configuração da página
st.set_page_config(page_title="CineIA", page_icon="🎬 ")
st.title("🎬 CineIA: Seu Próximo Filme")
st.markdown("Descubra filmes baseados no seu estado de espírito atual.")

# --- INTERFACE ---
with st.sidebar:
    st.header("Preferências")
    genero = st.multiselect("Gêneros favoritos:",
                            ["Ação", "Drama", "Sci-Fi", "Comédia", "Terror", "Documentário"])
    tempo = st.slider("Duração máxima (minutos):", 60, 240, 120)

mood = st.text_area("Descreva como você está se sentindo ou o que busca no filme:",
                    placeholder="Ex: Quero um filme de ficção científica com reviravoltas na história.")

botao_recomendar = st.button("Buscar Recomendações")

# --- LÓGICA DE PROCESSAMENTO ---
if botao_recomendar:
    if not mood:
        st.warning("Por favor, descreva o que você deseja assistir!")
    else:
        with st.spinner("Analisando catálogo cinematográfico..."):
            prompt = f"""
            Você é um especialista em cinema.
            Recomende 3 filmes para alguém que gosta de {', '.join(genero)} e que durem até {tempo}
            minutos.
            O usuário descreveu o clima do filme como: '{mood}'.
            Para cada filme, forneça: Título, Ano e uma frase curta do porquê combina com o pedido.
            """
            try:
                response = model.generate_content(prompt)
                st.success("Aqui estão minhas sugestões:")
                st.markdown("---")
                st.write(response.text)

# --- FEEDBACK COM BOTÕES E ÍCONES ---
st.markdown("#### Deixe seu feedback:")
col1, col2 = st.columns(2)

with col1:
    if st.button("👍 Gostei"):
        with open("feedback.csv", "a") as f:
            f.write(f"{mood},{genero},{tempo},Gostei\n")
        st.success("Obrigado pelo seu feedback positivo!")

with col2:
    if st.button("👎 Não gostei"):
        with open("feedback.csv", "a") as f:

```

```
f.write(f"{mood},{genero},{tempo},Não gostei\n")  
st.info("Feedback registrado. Vamos melhorar!")
```

except Exception as e:

```
st.error(f"Erro ao conectar com a IA: {e}")
```

*st.caption("Desenvolvido na disciplina de Desenvolvimento Full Stack – Sistema de Informação –
Universidade Franciscana (UFN)")*